GPU Accelerated Facial Recognition

Mason McGaffin

ECEN 5763 Embedded Computer Vision

Dr. Sam Siewert

University of Colorado Boulder

July 24, 2026

Contents

Introduction

The objective of this project was to investigate how GPU hardware acceleration can improve facial recognition on real-time embedded systems. The goal was to successfully detect and identify nine (9) individuals using a live webcam or prerecorded video while continuously processing on the NVIDIA Jetson Orin Nano.

The project began by exploring classical face detection and recognition methods. For detection, I explored Histogram of Oriented Gradients (HOG) and Haar Cascade for frontal face detection. For recognition, I explored methods including Local Binary Patterns (LBP), Eigenfaces, and Fisherfaces. After encountering roadblocks (discussed in Functional Design Overview), a pivot was made to explore deep learning methods such as YuNet (YN) detection and SFace (SF) recognition.

After multiple iterations, the final proof-of-concept product implements GPU accelerated deep learning based facial attendance operating in real-time. The application accepts input from prerecorded video and photos or live video camera input. The program is capable of processing at over 50 frames per second (FPS). For live video feed, the program was limited by the hardware which captured at 30 FPS; the output was displayed at the same rate.

Functional Requirements

Description	**Requirement**	**Result**
Accepts webcam input	The application shall accept live feed input from the Logitech C920X Pro HD Webcam.	✅ The user can specify the `--live` parameter to use live camera input from the webcam
Accepts mp4 input	The application shall accept input from prerecorded MP4 video.	✅ The user can specify the `--input` parameter to use a MP4 video as input
Save output	The application shall save the modified frames to JPEG files.	✅ The application saves frames to the “frames” directory as JPEG images. The user can specify another location to save them using the `--outdir` parameter.
Face detection	The application shall reliably detect front facing persons.	✅ The system detects and boxes detected faces.
Face recognition	The application should reliably recognize the identities of nine (9) individuals.	✅ The system labels recognized faces or applies a “STRANGER” label if not recognized.
Performance tracing	The application shall log critical timing events to conduct post-execution performance analysis.	✅ The system incorporates a TraceLogger that saves events in a packed struct to write-back to a CSV file at the end of the execution.

		Python scripts can be used to parse and create visualizations of the execution performance.
Feasible Frame Rate	The application shall maintain a feasible frame rate	✅ With MP4 video input, the program can achieve over 50 FPS. With live video input, the C920 has a max frame rate of 30 FPS which limits the output frame rate to 30 FPS.
Feasible Memory Requirements	The application shall utilize less than 3 GB of RAM.	✅ The application properly caps queue sizes. The maximum RAM utilization is ~800MB.
CPU Utilization	The system shall utilize multiple CPU cores using multi-threading	✅ The system uses separate threads for the input, processing, and write-back tasks.
GPU Utilization	The system shall utilize the GPU to accelerate frame processing	✅ The system uses GPU accelerated deep learning model predication to identify faces.
Stable Execution	The system should not drop frames and maintain consistent execution times	✅ The processing execution time is relatively consistent after the initial frame due to GPU activation latency. ❌ The write-back execution time is susceptible to blocking from the Jetson SD Card which causes latency spikes.

Machine Vision Requirements

Description	**Requirement**	**Requirement**
Face detection	The application shall reliably detect one or more front facing persons.	✅ The system uses a YuNet detection model from OpenCV to detect human faces in the frame. Precision: Recall:
Face localization	The application shall produce accurate bounding boxed around detected faces.	✅ The application produces green boxes around recognized faces and red boxes around not recognized faces.
Face recognition	The application should reliably recognize the identities of nine (9) individuals.	✅ The system trains a SFace recognition model from OpenCV used to identify faces in the frame.

		Precision: Recall:
Face labeling	The application shall produce text indicating the predicted face detected and the confidence in the prediction.	✅ The application produces green text with the subject's name and prediction confidence [0-1] if recognized or labels them as a stranger if not.
Multi-Person Support	The application shall be able to detect and recognize multiple faces in a single frame	✅ The application can detect, recognize, and label two or more people in a singular frame.
Robustness	Facial recognition should be resistant to moderate changes in lighting, facial expression, motion, and accessories	✅ The deep learning model allows for more sophisticated processing that can confidently recognize individuals in variable conditions.

Functional Design Overview

Design Iteration

The first attempts at solving the real-time facial recognition problem were implemented using classical computer vision techniques. In Exercise #5, it was demonstrated that Haar Cascade can be accelerated using CUDA for face detection and that LBP can be used to identify individuals in static images. However, this simple design was limited in terms of performance (even with multi-threading) and accuracy. The LBPHFaceRecognizer struggled to account for motion and expression and lighting changes and was also compute intensive. In order to improve accuracy, an ensemble model was attempted. The model combined three OpenCV face recognition models: LBP, Eigenfaces, and Fisherfaces to create a more stable prediction by eliminating individual biases. However, the models struggled to agree and the confidence remained low. Although some improvements were noted, the accuracy remained unsuitable for a face recognition system. This led to further research being conducted and experimentation with deep-learning models. These models were selected for the Final Design because of the tremendous accuracy improvements and performance enhancements using GPU acceleration.

Final Design

The system operates continuously after an initial training stage. During the startup, the application initializes the YuNet face detection model and the SFace recognition model. Then, the application trains the recognition model using labeled data from raw images. Finally, the write back and input threads are initialized before continuous operation.

The main loop of the application uses three (3) threads – input, processing (main), and write-back. The block diagram below shows the data pipeline that each input frame follows from the input handler to the detection and recognition processing, and finally to the write-back handler. In between the threads are two bounded queues that allow other tasks to continue running if another become slowed down. This is especially important due to blocking that may arise from the SD Card driver when the writer thread is attempting to continuously write image files.

After the input handler receives all of the frames, as indicated by the user or the conclusion of the input video file, the application waits for processing and writeback to be complete before gracefully shutting down.

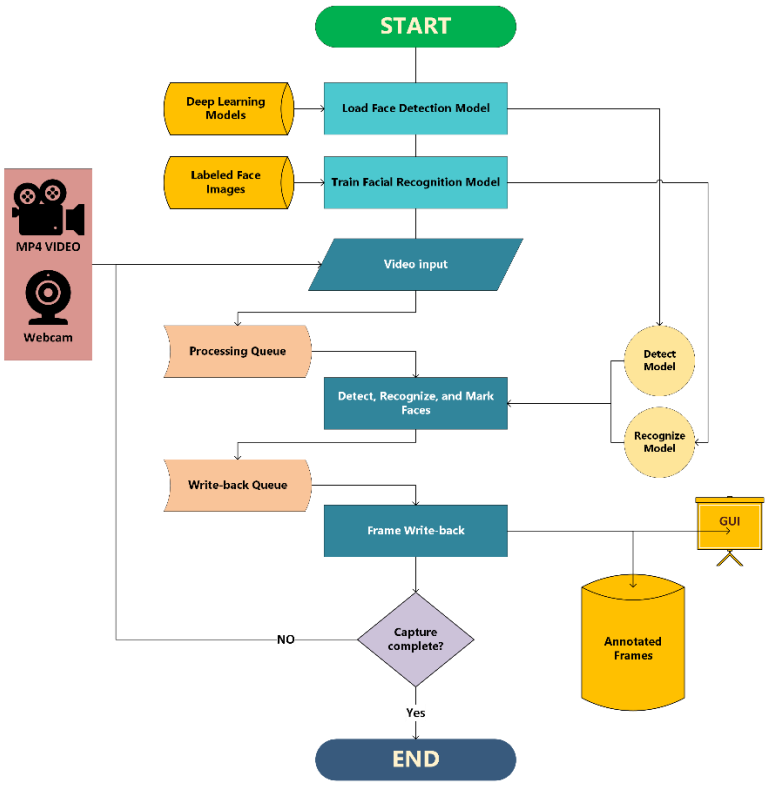

Figure XX Block Diagram

Input Thread

The input thread is responsible for continuously acquiring image frames from either a live webcam stream or a prerecorded video (or static image) file. The thread operates independently from recognition and output threads to prevent latency from blocking frame acquisition.

On initialization, it opens a webcam device or retrieves a video file using OpenCV's VideoCapture interface. The input thread continuously retrieves frames from the input source, indexing the frame, and logging the capture timestamp. The captured frame is stored in a shared queue directed to the processing thread. After pushing the frame into the queue, the input thread signals the processing thread using a condition variable.

```
while running:
      frame = read_frame();
      timestamp();
      in_queue.push(frame);
      notify();
```

Code XX: Pseudocode for Input Thread

For thread synchronization, the input handler uses a mutex and two condition variables. The locking prevents race conditions between the input and processing threads. There are two condition variables used. One is used to yield until space is available in the queue before pushing. This prevents frames from being dropped. The other is used to notify the processing thread when new frame is ready to be processed.

Separating frame acquisition into its own thread allows the system to continue receiving frames while computationally expensive face detection and recognition operations are performed. This improves throughput and prevents capture delays from reducing overall frame rate.

Processing Thread

The processing thread is responsible for performing the face detection and recognition. This thread is notified by the input thread when a new frame is ready to be processed. The processing thread pops the next frame and uses the face detection model to detect faces in the frame. Each face is sent to the face recognition model to be aligned and identified against the database that the model was trained on. If the model can confidently identify the face, it draws a bounding box around the face with text displaying the individuals name and the models confidence in that label. If a face is not recognized as an individual in the database, the bounding box is red and the individual is marked as a “stranger”. The annotated frame is then unloaded to the writer thread.

```
while running:
      if new_frame:
            timestamp();
            frame = in_queue.pop();
            faces = detect_faces(frame);
            for (face in faces) :
                  aligned_face = align_face(face);
                  label = predict(aligned_face);
                  draw_label(frame, label);
            timestamp();
            out_queue.push(frame);
```

Code XX: Pseudocode for Processing Thread

As discussed before, the processing thread is synchronized with the input thread using a condition variable and mutex pair. When a new frame arrives, the processing thread dequeues it, processes and annotates the frame, pushes the annotated frame to the output queue, and notifies the writer thread that there is a frame ready.

Writer Thread

The writer thread is responsible for saving the annotated frames to disk. The writer settings can be configured by the user to set output directory location and whether to use a live display. After being notified by the processing thread, the writer dequeues the frame and performs the output functions based on the user's input. By default, the writer will save the frame as a JPEG image, labelling it with the frame index.

```
while running:
      if new_frame:
            timestamp();
            frame = out_queue.pop();
            if (save):
                  save_frame(frame);
            if (GUI):
                  show_frame(frame);
            timestamp();
```

Code XX: Pseudocode for Writer Thread

The writer thread is synchronized with processing thread using condition variables and a mutex. Similar to the input thread, the two condition variables are used for indicating when a frame has entered the queue and when there is no space left in the queue. These signals ensure safe file writing and ensure no frames are dropped.

It is very important that the writer thread is on its own thread because as we have seen frequently in this class, occasionally the SD Card driver will block when continuous write operations are executed. This causes temporary write-back latency. By separating this task, the upstream tasks of input capture and frame processing can continue without blocking. It also allows the writer to catch back up on saving frames after the SD Card blocking is lifted.

Machine Vision Analysis

Dataset

The goal of the project was to successfully be able to distinguish the identity of nine (9) individuals and also recognize a person who is not within the training group. To create the training data set, I collected between six (6) and eight (8) portraits of nine (9) individuals including myself. The photos included different facial expressions and a hat accessory. These images were preprocessed to a fixed size, and naming conventions were used to label the images. During the training, the face detection model detects the face, converts to grayscale, and crops and scales the region of interest (ROI) toa 200x200 image. The recognition model intakes each cropped headshot labeled with an id [1-9]. Each ID is mapped to the subject's name. Each user is registered in a simple database with feature embeddings to be used for comparison during inference.

Figure XX. Raw image (left) and cropped/scaled training image (right)

Challenge Set

The challenge set consists of two (2) MP4 videos and nine (9) headshots of individuals wearing glasses, and one group photo. The headshots were used as a simple benchmark for recognition whereas the MP4 videos represented the all encompassing challenge of building a real-time system resistance to changes in lighting, position, and expression. The videos contain footage of different individuals entering and exiting the frame with changing facial positions and expressions. The group photo contains seven (7) individuals to investigate the response to multiple faces in the frame. As discussed in the design overview, the system also allows for webcam video input. The live recorded footage shows myself and a stranger (my roommate who was not included in the training data). This was intended to show the model can distinguish between known and unknown individuals.

Detection Analysis

In early iterations, the Haar Cascade classifier was used to detect faces within the frame. As documented in Exercise #5, this classifier is able to be sped up using the built-in CUDA target. The GPU acceleration contributed about a 40% speedup for detection. However, Haar Cascade out of the box is imprecise. It gives a rough bounding rectangle of the detected face and does not use alignment that modern tools offer. This impacts the accuracy of the subsequent recognition model because the inference data is not uniform to the trained data.

The face detection algorithm used in the final deep-learning product is the FaceDetectorYN API. A predefined model (ONNX) can be loaded and run directly in OpenCV using the DNN (Deep Neural Networks) module. The model is comprehensive and can detect faces from multiple angles more than just the front facing position. The YuNet model is a lightweight and ultra-fast

deep-learning face detection model. It contains only about 75,000 parameters making it ideal for resource constrained embedded systems. The model can be run with the default OpenCV backend and CPU target or an optimized NVIDIA CUDA backend and GPU target. The performance tradeoff is further discusses in Embedded Performance Analysis.

The following convolution matrices were evaluated based on each frame of the deep-recognizer application output. The frames were treated as all or nothing – if any false recognition (positive or negative occurred), that took precedence.

Challenge Video 1: 804 Frames

	T	F
P	535	33
N	316	0

Table XX: Convolution Matrix for Detection Accuracy in Challenge Video 1

Challenge Video 2: 501 Frames

	T	F
P	283	2
N	215	1

Table XX: Convolution Matrix for Detection Accuracy in Challenge Video 2

Webcam Video: 500 Frames

	T	F
P	498	3
N	0	0

Table XX: Convolution Matrix for Detection Accuracy in Webcam Capture

Group Photo: 1 Frame – 7 Faces

	T	F
P	1	0
N	0	0

Table XX: Convolution Matrix for Detection Accuracy in Group Photo

Combined Overall:

	T	F
P	1317	38
N	531	1

Table XX: Convolution Matrix for Overall Detection Accuracy

$$P = \frac{1317}{1317 + 38} = 0.972 \qquad R = \frac{1317}{1317 + 1} = 0.999$$

Recognition Analysis

Recognition was the biggest hurdle of the project. There were tradeoffs between accuracy and performance with classical models. The deep-learning based model addressed both of these barriers.

Initially, the classical LBPHFaceRecognizer from OpenCV was the target model to be used for the facial recognition. LBPH builds upon the texture operator of LBP to specialize in facial recognition. Local Binary Patterns (LBP) is a texture operator that characterizes the spatial structure of an image using logical comparisons for each pixel to the eight (8) neighboring pixels, resulting in binary representations. LBPH introduces histograms (H) that divide the binary texture image to be concatenated into a single feature vector. The vector preserves both textures and spatial locations to recognize distinct facial features. Inference histograms are compared against the trained data using distance algorithms (Euclidean distance) to produce a label and confidence metric. However, the variable conditions of the challenge video made it difficult for the model to distinguish between trained individuals. It resulted in many false positive and false negative results.

The next attempt to improve the accuracy of the recognition model was to use an ensemble model. Ensemble models are a machine learning technique that combine multiple individual models – in this case LBP, Eigenfaces, and Fisherfaces from OpenCV – into a single framework to improve accuracy and generalization. Eigenfaces is a computer vision model that uses the eigenvectors of a covariance matrix to produce extracted “Eigenfaces”. Inference faces are projected onto the trained Eigenfaces using a dot product to create a weighted vector used to label the image. Fisherfaces uses Linear Discriminant Analysis (LDA) to find maximum class separability. It is more resistant to lighting changes which was intended to balance out the susceptibility from LBP and Eigenfaces. However, the ensemble model was not successful. The individual models frequently disagreed on the face label and were still very susceptible to lighting and orientation changes. LBP and Fisherfaces require significant amounts of training data to be resistant to changing light, facial expressions, and orientation which were unviable for the scope of the project.

The next and final iteration was using a deep-learning model. The FaceRecognizerSF API is a highly optimized backend that enables deep learning models to be run directly on OpenCV. The SFace model is an accurate, ultra-fast recognition mode optimized for edge processing. The model works by projecting facial feature vectors onto a normalized multi-dimensional hypersphere. The model uses MobileFaceNet backbone to extract embedding vectors from aligned facial images. The model can be run using a CPU or GPU backend and target. As described in the Embedded Performance Analysis section, the CUDA GPU acceleration improved performance by about 100%.

The following convolution matrices are developed from each of the positive face detections (true or false) in the detection step. There were frames with multiple faces detected. Constrasting to

the face detection analysis, each face is analyzed individually – if a frame has two faces, the recognition result for each face is unique.

Challenge Video 1: 568 frames with detected faces

	T	F
P	466	0
N	30	78

Table XX: Convolution Matrix for Recognition Accuracy in Challenge Video 1

Challenge Video 2: 285 frames with detected faces

	T	F
P	262	1
N	2	28

Table XX: Convolution Matrix for Recognition Accuracy in Challenge Video 2

Webcam Video: with detected faces

	T	F
P	582	4
N	415	24

Table XX: Convolution Matrix for Recognition Accuracy in Webcam Capture

Group Photo: 7 faces

	T	F
P	7	0
N	0	0

Table XX: Convolution Matrix for Recognition Accuracy in Group Photo

Combined Overall:

	T	F
P	1317	5
N	447	130

Table XX: Convolution Matrix for Overall Recognition Accuracy

$$P = \frac{1317}{1317 + 5} = 0.996 \qquad R = \frac{1317}{1317 + 130} = 0.910$$

Embedded Performance Analysis

Initial Constraints

During initial implementations using LBP or the ensemble model, not only did the accuracy struggle but the performance was quite limited. As displayed below in **FIGURE XX**, the execution time to detect (HaarCascadeClassifier) and recognize (LBPHFaceRecognizer) a 640x480 pixel frame averaged ~93ms. Assuming no other latency in the system, that would cap the processing rate at ~11 FPS.

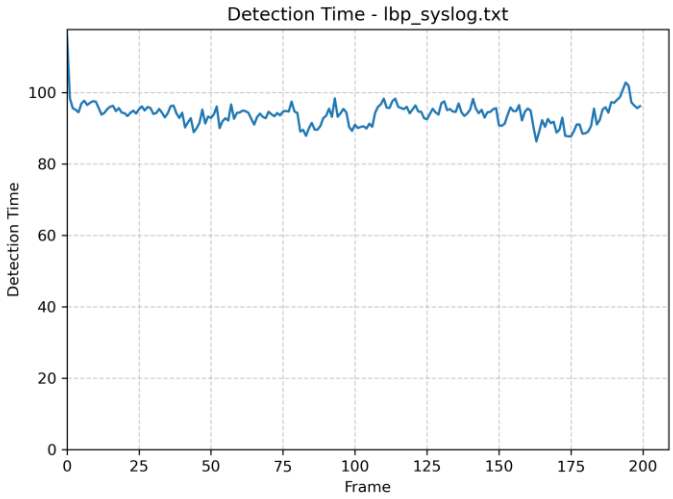

Figure XX: Execution time for simple detect and recognize using Haar Cascade and LBPH

Introducing a multi-threaded ensemble model improved accuracy somewhat. However, it reduced performance even more. Even when using OpenMP to parallelize processing threads, the resulting FPS was still limited to under 10 FPS. It was clear that more parallelization, a different detection/recognition model, and GPU acceleration is needed.

The Jetson Orin Nano has six cores to more CPU parallelization was not feasible for the application. However, the performance improvements came from using a deep-learning model. The following charts compare the performance of a strict CPU implementation and a CUDA accelerated implementation that uses the GPU.

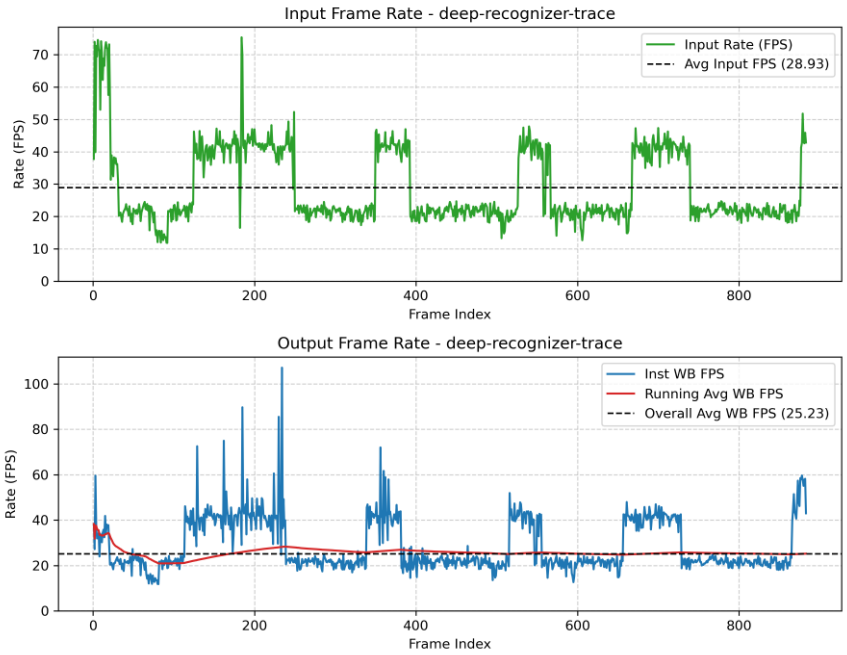

Figure XX Input and Output Frame Rate for CPU-only implementation (Challenge Video 1)

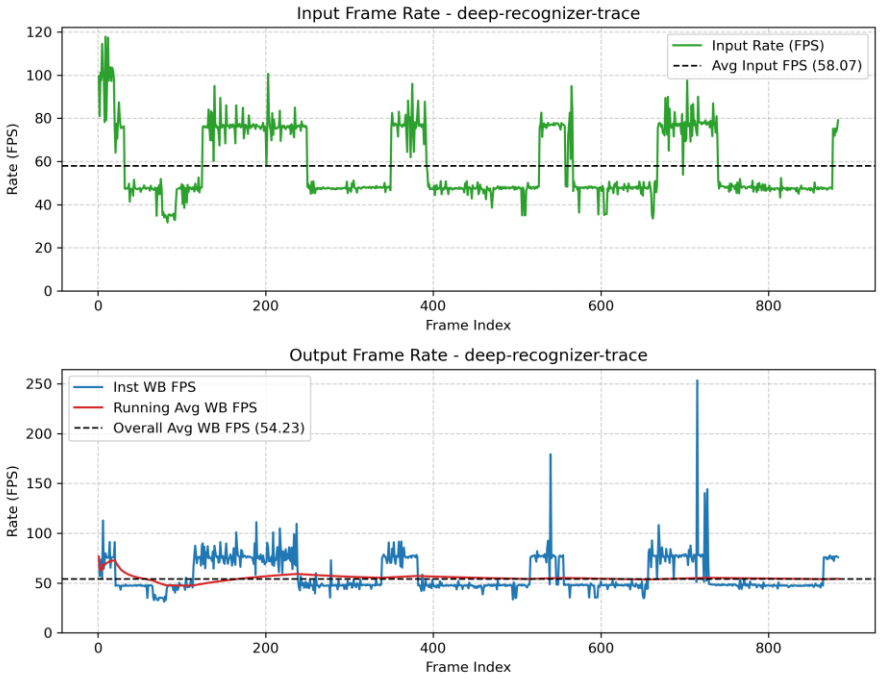

Figure XX Input and Output Frame Rate for CUDA implementation (Challenge Video 1)

As seen in the graphs above, GPU acceleration enables the output frame rate to increase by 100% from ~25FPS to over 54FPS.

The final product that implements CUDA acceleration can be summarized with the following performance characteristics table.

	Metric				
Input Media	**Input FPS**	**Output FPS**	**CPU Usage**	**GPU Usage**	**RAM (MB)**
Challenge Video 1	58.07	54.23	~72%	~85%	~1000
Challenge Video 2	59.08	51.56	~70%	~87%	~700
Webcam	30	30	~40%	~85%	~700

The Appendix contains more figures that demonstrate the performance of the embedded application.

Proof-of-Concept

As discussed in the requirements sections, the main project goal has been achieved. The application is able to successfully identify nine (9) unique individuals with feasible performance metrics. Below are captured frames from each of the challenge media showing the achievement of the project goal of facial recognition and labeling for multiple people. The Appendix contains zip files to example results.

Figrue XX

Figure XX

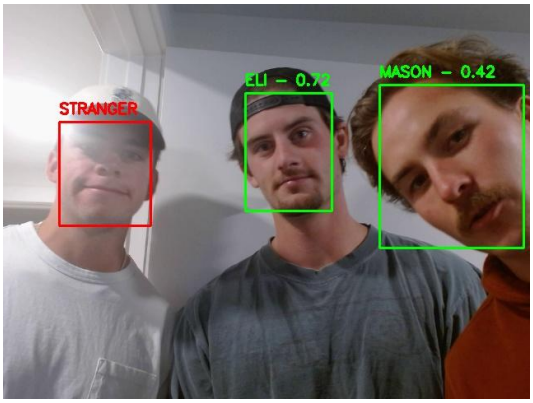

Figure XX

As seen in the captures above, eight (8) indiivudals are uniquely identified. One person in the dataset, (person 1, Jonny) was unavailable for the challenge video recording. Also observe in Figure XX that a stranger is idenitified. This is Alex, my roommate who was out of town while gathering training data. His inclusion in the study was to demonstrate the application can not only identify known persons but mark unknown persons as strangers. See the Appendix for links to more frames and MP4 video solutions.

Conclusion

The project evolved through three generations of computer vision pipelines. Initial experiments with Haar Cascade detection and LBPH recognition established a functional baseline but revealed limitations in both recognition accuracy and computational performance. An ensemble of LBPH, Eigenfaces, and Fisherfaces improved classification robustness but introduced additional computational overhead while remaining dependent on classical face detection. The final implementation adopted the OpenCV DNN deep-learning YuNet detector and SFace recognizer, which significantly improved both detection accuracy and real-time performance on the Jetson platform. The resulting multithreaded system met the project's functional and performance objectives while demonstrating stable continuous operation.

Attributions

I would like to explicitly recognize and thank the following people for their role in making this project possible. I would like to thank Dr. Sam Siewert for his guidance, mentorship, and teaching. I would like to thank my friends Jonny Patel, Eli Swerland, Rowen Kennedy, Nate Kopec, Cy Sokolowski, Mason Hibbett, and James Johnson for being participants in my data collection and being supportive of my effort in this project. I would also like to thank my family for their continuous support throughout my Academic career. Without them, I would not be in the position I am today to pursue a higher education.

References

[1] Cppreference, "cppreference.com — C++ Reference." [Online]. Available: https://en.cppreference.com/

[2] T. G. Dietterich, "Ensemble Methods in Machine Learning," in *Multiple Classifier Systems* (Lecture Notes in Computer Science), vol. 1857, J. Kittler and F. Roli, Eds. Berlin, Heidelberg: Springer, 2000, pp. 1–15, doi: 10.1007/3-540-45014-9_1.

[3] S. Gaur, M. Pandey, and Himanshu, "Realization of Facial Recognition Technology for Attendance Monitoring Through Biometric Modalities Employing MTCNN Integration," *SN Comput. Sci.*, vol. 5, no. 7, art. no. 732, 2024, doi: 10.1007/s42979-024-03225-1.

[4] H. Jun, C. Jian-feng, F. Ling-zhi, and H. Zhong-wen, "A method of facial expression recognition based on LBP fusion of key expressions areas," in *Proc. 27th Chinese Control Decis. Conf. (CCDC)*, Qingdao, China, 2015, pp. 4200–4204, doi: 10.1109/CCDC.2015.7162668.

[5] JetsonHacks, "OpenCV 4 + CUDA on Jetson Nano," *JetsonHacks*, Nov. 22, 2019. [Online]. Available: https://jetsonhacks.com/2019/11/22/opencv-4-cuda-on-jetson-nano/

[6] A. Mallick, "Face Recognition Models: A Complete Guide," *LearnOpenCV*, Feb. 2, 2023. [Online]. Available: https://learnopencv.com/face-recognition-models/

[7] NVIDIA Corporation, "Monitoring Tools — Tegrastats Utility," *NVIDIA Jetson Linux Developer Guide*, Rel. 36.4, 2024. [Online]. Available: https://docs.nvidia.com/jetson/archives/r36.4/DeveloperGuide/AT/JetsonLinuxDevelopmentTools/TegrastatsUtility.html

[8] OpenCV, "OpenCV: Open Source Computer Vision Library," GitHub repository. [Online]. Available: https://github.com/opencv/opencv

[9] OpenCV, "OpenCV Model Zoo," GitHub repository. [Online]. Available: https://github.com/opencv/opencv_zoo

[10] OpenCV Content Team, "OpenCV 4.10.0 Documentation," 2024. [Online]. Available: https://docs.opencv.org/4.10.0/

[11] OpenCV Content Team, "cv::FaceDetectorYN Class Reference," *OpenCV 4.10.0 Documentation*, 2024. [Online]. Available: https://docs.opencv.org/4.10.0/df/d20/classcv_1_1FaceDetectorYN.html

[12] OpenCV Content Team, "cv::FaceRecognizerSF Class Reference," *OpenCV 4.10.0 Documentation*, 2024. [Online]. Available: https://docs.opencv.org/4.10.0/da/d09/classcv_1_1FaceRecognizerSF.html

[13] OpenCV Content Team, "cv::face Namespace Reference," *OpenCV 4.10.0 Documentation*, 2024. [Online]. Available: https://docs.opencv.org/4.10.0/d4/d48/namespacecv_1_1face.html

[14] PlantUML, "PlantUML — Draw UML diagrams from plain text." [Online]. Available: https://www.plantuml.com/plantuml

[15] M. S. E. Saadabadi, S. R. Malakshan, S. R. Hosseini, and N. M. Nasrabadi, "Boosting Unconstrained Face Recognition with Targeted Style Adversary," *arXiv:2408.07642 [cs.CV]*, Aug. 2024, doi: 10.48550/arXiv.2408.07642.

[16] S. Siewert, "Lecture Week 6-1: Discussion Documents," *CSCI 682 Course Materials*, California State University, Chico. [Online]. Available: https://www.ecst.csuchico.edu/~sbsiewert/csci682/documents/Discussions/Lecture-Week-6-1.pdf

[17] S. Siewert, "Lecture Week 6-2: Discussion Documents," *CSCI 682 Course Materials*, California State University, Chico. [Online]. Available: https://www.ecst.csuchico.edu/~sbsiewert/csci682/documents/Discussions/Lecture-Week-6-2.pdf

[18] S. Siewert, "Tutorial: CUDA and Multi-Core," *ECV-5763-CU Lecture Documents*, ESEE Web Resources, University of Colorado Boulder. [Online]. Available: https://o365coloradoedu-my.sharepoint.com/

[19] V. Srikrishnan, "Getting Started with OpenCV CUDA Module," *LearnOpenCV*, Sep. 20, 2020. [Online]. Available: https://learnopencv.com/getting-started-opencv-cuda-module/

[20] YouTube, "OpenCV / CUDA Video Processing Demonstration," YouTube video. [Online]. Available: https://www.youtube.com/watch?v=sz25xxF_AVE

Appendix

GitHub repository: https://github.com/mamc3334/ECV_5763

Pull request: https://github.com/mamc3334/ECV_5763/pull/7

Downloadable Zip Files:


example-video.zip


Repo.zip

Repo breakdown:

- The final proof-of-concept application lives in Project/Samples/deep-learning
- The results from the various challenges contain all PNG images of performance and timing charts
- There are ZIP files of the frames used to create MP4 videos for each challenge in example-video.zip
- All raw and modified training data is included in the data directory.
- All challenge videos and photos are in the challenge directory